
Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computación

Compiladores e Interpretes

Semestre I, 2026

Prof. Allan Rodríguez Dávila

Proyecto #3

Generación de Código Destino (MIPS)

Estudiantes:

Owen Smith Cerdas

Keyner Cerdas Morales

1. Descripción del Problema

Un grupo de desarrolladores requiere un lenguaje imperativo ligero orientado a la configuración de chips. La industria de sistemas empotrados demanda herramientas de compilación capaces de traducir código de alto nivel a instrucciones de máquina eficientes, razón por la cual el proyecto solicita el desarrollo completo de un compilador que incluya análisis léxico, sintáctico, semántico, generación de código intermedio de tres direcciones y, como fase final, la generación de código MIPS ejecutable en QtSpim.

Este proyecto (Proyecto #3) comprende específicamente la fase de generación de código destino. Se toma como base el front-end construido en los Proyectos I y II (scanner JFlex, parser CUP, analizador semántico y generador de código intermedio) y se añade el módulo de traducción a MIPS. Todo programa escrito en el lenguaje debe contener exactamente un método main y una secuencia de declaraciones de procedimientos con distintas expresiones.

2. Manual de Usuario

2.1 Requisitos Previos

- Java JDK 11 o superior instalado y configurado en el PATH del sistema.
- Librerías JFlex (jflex-full-1.9.1.jar) y CUP (java-cup-11b.jar, java-cup-11b-runtime.jar) en la carpeta lib/.
- QtSpim instalado para la verificación de la ejecución del código MIPS generado.
- Sistema operativo: Windows, macOS o Linux.

2.2 Estructura del Proyecto

El proyecto se organiza de la siguiente manera:

```
PP3.zip
├── info.txt
├── documentacion/
│   └── Documentacion_Externa.docx
└── programa/
    ├── src/
    │   ├── main/Main.java
    │   ├── parser/Parser.cup
    │   ├── parser/mipsGenerator.java
    │   ├── parser/cod3D.java
    │   ├── scanner/Lexer.flex
    │   ├── tabla/Tabla.java
    │   ├── tabla/TablaManagement.java
    │   └── tabla/Simbolo.java
    ├── lib/
    │   ├── jflex-full-1.9.1.jar
    │   ├── java-cup-11b.jar
    │   └── java-cup-11b-runtime.jar
    └── src/tests/prueba.txt
```

2.3 Compilación

Ejecutar los siguientes comandos desde la raíz del proyecto (carpeta programa/):

Paso 1 — Generar el scanner con JFlex:

```
java -jar lib/jflex-full-1.9.1.jar src/scanner/Lexer.flex
```

Paso 2 — Generar el parser con CUP:

```
java -jar lib/java-cup-11b.jar -parser parser -symbols sym -expect 6 -destdir
src/parser src/parser/Parser.cup
```

Paso 3 — Compilar el proyecto Java:

```
javac -d out -cp "lib/java-cup-11b-runtime.jar" src/main/*.java src/parser/*.java  
src/scanner/*.java src/tabla/*.java
```

2.4 Ejecución

Ejecutar el compilador pasando como argumento la ruta al archivo fuente:

En macOS / Linux:

```
java -cp "lib/java-cup-11b-runtime.jar:out" main.Main src/tests/prueba.txt
```

En Windows:

```
java -cp "lib/java-cup-11b-runtime.jar;out" main.Main src/tests/prueba.txt
```

Nota: el separador del classpath en Windows es ; mientras que en macOS y Linux es :. El archivo fuente puede ser cualquier archivo de texto válido escrito en el lenguaje del compilador.

2.5 Archivos Generados

Tras la ejecución, el compilador genera o actualiza los siguientes archivos en la carpeta raíz del proyecto:

Archivo	Descripción
tokens.txt	Lista completa de tokens reconocidos por el scanner.
lexical_errors.txt	Errores léxicos detectados durante el análisis.
syntax_errors.txt	Errores sintácticos reportados por el parser con recuperación en modo pánico.
semantic_errors.txt	Errores semánticos detectados durante el análisis.
tabla_simbolos.txt	Tabla de símbolos exportada con scope, nombre, tipo, categoría, línea y columna.
cod3D.txt	Código intermedio de tres direcciones generado.
codMips.asm	Código MIPS destino listo para ejecutar en QtSpim.

2.6 Verificación en QtSpim

- Abrir QtSpim en el sistema operativo correspondiente.
- Seleccionar File → Load y cargar el archivo codMips.asm generado.
- Verificar que no se presenten errores de sintaxis MIPS en la consola de QtSpim.
- Ejecutar el programa con el botón Run o mediante F5 y observar los resultados en la consola de salida.

3. Pruebas de Funcionalidad

3.1 Caso de Prueba Principal

El archivo utilizado para verificar el correcto funcionamiento del compilador es `src/tests/prueba.txt`. Este archivo contiene un programa escrito en el lenguaje imperativo del proyecto que ejercita las principales características del compilador: declaración de variables, arreglos, funciones, condiciones, ciclos, operaciones aritméticas (incluyendo potencia y módulo), manejo de flotantes, cadenas y la instrucción CIN.

3.2 Verificaciones Realizadas

- El scanner reconoce correctamente todos los tokens del archivo fuente y los exporta a `tokens.txt`.
- El parser valida la estructura gramatical del programa y reporta errores sintácticos con recuperación en modo pánico.
- El analizador semántico valida tipos, declaraciones, uso de arreglos, llamadas a funciones, condiciones y retornos.
- El generador de código intermedio produce `cod3D.txt` con las instrucciones de tres direcciones correctas.
- El generador MIPS traduce `cod3D.txt` a `codMips.asm` con secciones `.data` y `.text` bien formadas.
- El archivo `codMips.asm` se carga y ejecuta correctamente en QtSpim sin errores de ejecución.

3.3 Resultado Esperado

Al ejecutar el compilador sobre el archivo de prueba, se espera que:

- Los archivos de reporte (tokens, errores, tabla de símbolos) se generen con el contenido correcto.
- El archivo `cod3D.txt` refleje semánticamente las operaciones del código fuente.
- El archivo `codMips.asm` produzca la salida esperada al ejecutarse en QtSpim.

4. Librerías Usadas

El proyecto utiliza exclusivamente las siguientes librerías externas. No se requieren dependencias adicionales para compilar y ejecutar el proyecto.

Librería	Versión / Archivo	Propósito
JFlex	jflex-full-1.9.1.jar	Generación del analizador léxico a partir de Lexer.flex.
CUP (Constructor of Useful Parsers)	java-cup-11b.jar	Generación del analizador sintáctico a partir de Parser.cup.
CUP Runtime	java-cup-11b-runtime.jar	Soporte en tiempo de ejecución requerido para el parser generado por CUP.
Java Standard Library	JDK 11+	Manejo de archivos (FileReader, PrintWriter, FileWriter), colecciones y estructuras de datos internas.
QtSpim	Simulador MIPS (externo)	Verificación de la ejecución del código MIPS generado. No es una dependencia de compilación.

5. Análisis de Resultados

5.1 Objetivos Alcanzados

- Análisis léxico funcional: el scanner JFlex reconoce correctamente palabras reservadas, operadores, literales, identificadores, delimitadores y tokens de error.
- Análisis sintáctico con recuperación de errores: el parser CUP valida la estructura gramatical del lenguaje y recupera errores en modo pánico, reportándolos con información de línea y columna.
- Análisis semántico: se validan tipos, declaraciones, uso de arreglos, llamadas a funciones, condiciones, retornos y estructuras de control.
- Generación de código intermedio: las expresiones y sentencias válidas producen instrucciones de tres direcciones almacenadas en cod3D.txt.
- Generación de código MIPS: el módulo mipsGenerator.java lee cod3D.txt, asigna registros, administra el stack frame y emite instrucciones MIPS en codMips.asm.
- Manejo de flotantes: soporte completo para operaciones con punto flotante usando registros \$f de MIPS.
- Manejo de potencia y módulo: operadores adicionales implementados tanto en el parser CUP como en el generador MIPS.
- Implementación de CIN: instrucción de entrada implementada y traducida correctamente a syscall MIPS.
- Manejo de cadenas en .data: las cadenas del programa se almacenan correctamente en la sección .data del código MIPS.
- Exportación de reportes: tokens.txt, lexical_errors.txt, syntax_errors.txt, semantic_errors.txt, tabla_simbolos.txt, cod3D.txt y codMips.asm se generan en cada corrida.

5.2 Objetivos No Alcanzados o Parcialmente Alcanzados

- Selección dinámica del nombre del archivo destino: el nombre de salida codMips.asm está fijo en la versión actual; no se permite al usuario especificar un nombre distinto en tiempo de ejecución.
- Interfaz de selección de archivo fuente: la interfaz de entrada es básica (argumento de línea de comandos) y no cuenta con un diálogo gráfico de selección de archivo.

5.3 Razones

El proyecto priorizó la integración correcta de todos los módulos del compilador (front-end y back-end) y la generación semántica precisa del código MIPS. El tiempo disponible se concentró en garantizar que el código generado se ejecute correctamente en QtSpim, especialmente en escenarios que involucran manejo del stack en retornos de función, registros flotantes y operaciones compuestas. Los aspectos de interfaz de usuario y parametrización dinámica de salidas se identifican como mejoras a implementar en versiones futuras del proyecto.

6. Detalle de Algoritmos Importantes

6.1 Recuperación de Errores Sintácticos (Modo Pánico)

El parser implementa recuperación de errores mediante producciones especiales que incluyen el token error de CUP. Cuando el parser detecta una secuencia inválida, activa el modo pánico: descarta tokens de la entrada hasta encontrar un token de sincronización (por ejemplo, ; o }). Esto permite continuar el análisis desde un punto válido y acumular el mayor número posible de diagnósticos en una sola corrida, en lugar de abortar al primer error encontrado.

6.2 Construcción y Gestión de la Tabla de Símbolos

La tabla de símbolos se implementa con una arquitectura de pila de scopes (TablaManagement). Cada vez que se entra a un bloque o función se apila una nueva instancia de Tabla con un identificador de scope incremental. Al salir del bloque, la tabla se desapila pero los símbolos permanecen en un historial global, lo que permite consultar información de scopes ya cerrados durante la exportación. La búsqueda de un símbolo recorre la pila desde el nivel más interno hasta el global, implementando la regla de ocultamiento de variables.

6.3 Generación de Código Intermedio de Tres Direcciones

La clase cod3D actúa como buffer acumulador. Las acciones semánticas del parser invocan nuevaEtiqueta() y nuevaTemporal() para generar etiquetas y temporales únicos, y appendCod3D() para agregar instrucciones al buffer. El método reemplazarLinea() permite backpatching sobre instrucciones de salto ya emitidas. Al finalizar el análisis, getCodigo() retorna el contenido completo del buffer que se escribe en cod3D.txt.

6.4 Traducción de Código Intermedio a MIPS

El generador MIPS (mipsGenerator.java) lee cod3D.txt línea a línea e identifica el patrón de cada instrucción (asignación, operación aritmética, comparación, salto, llamada a función, retorno, etc.). Para cada patrón emite las instrucciones MIPS equivalentes. El algoritmo administra un mapa de registros temporales y utiliza la pila (\$sp) para el manejo de marcos de activación. Los strings se almacenan en la sección .data y se referencian con etiquetas únicas. Los retornos de función restauran el frame pointer y el return address antes de ejecutar jr \$ra.

6.5 Manejo del Stack Frame en Funciones

Al generar el prólogo de cada función, el generador reserva espacio en la pila para el frame pointer (\$fp), el return address (\$ra) y las variables locales. Al generar el epílogo (instrucción return), el generador restaura \$fp y \$ra desde la pila antes de emitir jr \$ra. Se corrigió un defecto en el que el valor de retorno era sobrescrito antes de que pudiera ser leído por el llamador, garantizando la correcta propagación de valores entre funciones.

6.6 Soporte de Flotantes

Las operaciones con punto flotante utilizan los registros \$f0-\$f30 de MIPS. El generador detecta cuando un operando es de tipo float a partir de la anotación en el código intermedio y emite instrucciones de la familia .s (add.s, sub.s, mul.s, div.s). Las conversiones entre entero y flotante se realizan mediante las instrucciones mtc1 y cvt.s.w según corresponda.

7. Bitácora (Historial de Commits)

La siguiente bitácora se generó a partir del historial real de git del proyecto. Cada entrada incluye el identificador del commit, autor, fecha y hora, mensaje descriptivo y los archivos afectados.

Commit	Autor	Fecha y hora	Mensaje	Código afectado
7e50df4	Owen Smith	22/06/2026 23:43	Se exportan archivos del proyecto pasado + mipsGenerator base para generación MIPS desde cod3D.txt.	mipsGenerator.java, Parser.cup, Lexer.flex, tabla/*, Main.java, tests/*
9fe0bcb	Keyner Cerdas Morales	25/06/2026 14:25	Actualizar cambios del proyecto. Ajustes generales en el compilador y su fase de generación.	Main.java, Parser.cup, mipsGenerator.java, Lexer.flex, tabla/*
6fef2b6	Owen Smith	25/06/2026 19:17	Se agrega en main.java la creación del archivo codMips donde se exporta el código destino generado.	Main.java, mipsGenerator.java
2fcc897	Owen Smith	25/06/2026 19:20	Merge branch 'master'. Fusión de ramas sin cambio funcional directo.	Fusión de ramas
fadea94	Owen Smith	26/06/2026 15:03	Se agrega manejo de flotantes, optimización de temporales y generación de código para condicionales.	mipsGenerator.java, tests/prueba.txt
e45670c	Owen Smith	27/06/2026 00:20	Se agrega manejo de potencia y módulo en semántica y generación MIPS.	Parser.cup, mipsGenerator.java, parser.java, tests/*
24b0ca4	Owen Smith	27/06/2026 04:47	Arreglar errores con manejo de la pila en retornos; se corrige sobreescritura del valor de retorno.	mipsGenerator.java
e7f2faa	Keyner Cerdas Morales	27/06/2026 20:51	Arreglar errores con registros flotantes, asignación de strings al .data e implementación de CIN.	Parser.cup, mipsGenerator.java, parser.java, tests/*
eafe70d	Keyner Cerdas Morales	27/06/2026 21:18	Implementación de cambios menores. Ajustes finales en la especificación CUP y clases generadas.	Parser.cup, parser.java, sym.java

Para ver el historial completo de git con grafo de ramas, ejecutar:

```
git log --oneline --decorate --graph --all
```